BAT

Coding standard

2 DOCUMENT HISTORY 4
3 APPLICATION RULES 5
4 FILES ORGANIZATION 6
4.1 MODULES 6
4.1.1 MODULES NAMING 6
4.1.2 FILE HEADER 7
4.1.3 LENGTH OF LINES 7
4.1.4 TABULATIONS 7
4.2 COMMENTS 7
4.3 HEADER FILE 8
4.4 FUNCTIONS 9
4.5 DEFINITIONS 10
4.6 SECTIONS 10
4.7 INTERNAL COMMENTS ON FUNCTIONS 10
5 MACRO 11
5.1 MACRO DEFINITIONS 11
5.2 HEADER HAT 13
6 TYPE DEFINITIONS 13
7 VARIABLES AND CONSTANTS 14
7.1 STANDARD VARIABLES DATATYPE 14
7.2 CONSTANTS DEFINITIONS 15
7.3 DEFINITION OF VARIABLES 15
8 FUNCTIONS 16
8.1 GLOBAL FUNCTIONS 16
8.2 LOCAL FUNCTIONS 16
8.3 FUNCTION CONTENT 17
8.4 FUNCTION PARAMETERS 17
9 CODING STYLE 18
10 OTHER PRESCRIPTIONS 19
11 TEMPLATES 20
11.1 FILE HEADER 20
11.2 FILE SOURCE 21
12 MEMORY & RESOURCE MANAGEMENT 21
12.1 DYNAMIC MEMORY ALLOCATION 22
12.2 RESOURCE CLOSING 22
12.3 BUFFER, ARRAY AND STRING BOUNDS 23
12.4 FORMAT STRINGS 23
13 SECURITY PRACTICES 23
13.1 CRYPTOGRAPHY AND TLS 23
13.2 SECRETS AND CREDENTIALS 24
13.3 BANNED AND INSECURE LIBRARY FUNCTIONS 24
13.4 FILESYSTEM AND ENVIRONMENT SECURITY 24
13.5 SENSITIVE DATA HANDLING 25

14 POINTER & NULL SAFETY 25
14.1 RETURNING OR RETAINING POINTERS TO LOCAL STORAGE 25
14.2 NULL POINTER DEREFERENCE 26
14.3 POINTER ARITHMETIC AND CASTING 26
14.4 POINTER INDIRECTION DEPTH 27
15 ERROR HANDLING & RETURN VALUES 27
15.1 RETURN VALUE CHECKING 27
15.2 PRECONDITIONS AND INITIALIZATION 28
15.3 PROGRAM TERMINATION AND NON-LOCAL JUMPS 28
15.4 FILE STREAM POSITIONING 29
16 STANDARD COMPLIANCE / UNDEFINED BEHAVIOUR 29
16.1 SIDE EFFECTS IN UNSEQUENCED EXPRESSIONS 29
16.2 ONE-DEFINITION RULE AND DUPLICATE DECLARATIONS 30
16.3 UNDEFINED AND UNSPECIFIED BEHAVIOUR 30
17 PREPROCESSOR SAFETY 31
17.1 #INCLUDE HYGIENE 31
17.2 CONDITIONAL COMPILATION DIRECTIVES 31
17.3 #PRAGMA USAGE 32
18 CONCURRENCY & THREAD SAFETY 33
18.1 MUTEX LIFECYCLE 33
18.2 REENTRANCY AND BLOCKING CALLS 33
19 ARITHMETIC & TYPE CONVERSION SAFETY 34
19.1 SIGNED / UNSIGNED MIXING 34
19.2 OVERFLOW AND WRAPAROUND 34
19.3 IMPLICIT CONVERSION AND CASTING PRECISION 34
19.4 OPERATOR USAGE AND PRECEDENCE 35
19.5 LITERALS, ESCAPE SEQUENCES AND STRING CONCATENATION 35
19.6 ENUMERATIONS AND LOOP COUNTERS 35
20 APPENDIX 36

1 Scope

The purpose of this document is to provide guidelines to standardize the style of the source code, so that the reading, verification and maintenance of the code written by different developers will be as transparent as possible.

Developing software following common standards provides several advantages over custom code writing. FW / SW designers have less difficulty in analyzing software written by others and the reuse of the code in new projects becomes faster.

Software written with shared rules offers a number of advantages such as fewer bugs and more efficient execution.

Furthermore, maintenance is the main cost driver in the software life cycle: it will therefore be easier for both the initial developer and others to modify and correct the sources as needed. The overall cost of the software is heavily reduced when the code is developed and managed according to established standards.

2 Document history

Date	Rev	Author	Approved	Note
25/11/2021	1.0	P. Pariani R. Pellegrini		1st issue
06/12/2021	1.1	P. Pariani		Updated § 8: special functions naming Added: §8.4 Function parameters Added: §**Error! Reference source not found.** Other p rescriptions**Error! Reference source not found.**
20/08/2026	1.2	Meghna P Andrew Tung		Added: 12 to 19 sections Conflicts and errors update

3 Application rules

Rules described in this document are mainly focused on software developed for embedded devices which generally are characterized by limited resources.

These rules must be applied to new developments, it is not necessary to rewrite old projects according to the style in question. But when making changes to an existing software project, the new standard must be followed for any new additions / changes made.

The rules indicated in this document are divided into standards and guidelines. A standard must be followed, and in case of violations, the developer must comment them in the code.

4 Files organization

A project is made up of several modules. Each module must be represented by a header file (usually an .h) and one or more source files (usually .c). Each module must perform a specific task and this task must be indicated in the name of the related files.

Project files can be organized into subdirectories by grouping modules by functional affinity.

4.1 Modules

Each module performs a specific function. Different functionalities must be implemented using different modules. Generally speaking, a module consists of a header file and a source file. In case a module implements many features, it is possible to split the source file into multiple files. All source files will refer to the same main module header file. If there are definitions common to the various source files that are not needed externally to the modules, it is possible to use header files inside the module. However, the main header file will remain unique, and this will contain all the definitions necessary for the use of the module by other modules.
Both the source file and the header file are divided into sections which will be defined later.

4.1.1 Modules naming

The name of the header and source files contained in a module must have the same name. The name of the module must be explanatory of its functionality.
The file name must not contain spaces or special characters. Only lowercase letters, numbers, and underscores are allowed, except that abbreviations may use uppercase letters as specified below. If the module name is made up of several words, the words must be separated by the '_' character to increase readability. All words must have lowercase letters, except abbreviations. The file extension must be lowercase.

Ex: *graphics.c / graphics.h*
display_utility.c / display_utility.h

If the name contains abbreviations, it is possible to use all capital letters for the abbreviations.

Ex: *ADC.c / ADC.h* ← → *adc.c / adc.h* both are correct.

4.1.2 File header

The header of each module file, both source and header, has a common structure. This structure includes the module description and copyright notices. The main source and header files of a module will have the same description. Header templates are inserted into the section 11.

4.1.3 Length of lines

In general, having too long lines of code makes them difficult to read and interpret. In principle, length should be kept within 80 columns, but since sometimes breaking a line of code makes it less intuitive, this rule can be violated, however remaining within 120 columns.

4.1.4 Tabulations

Source code shall be indented using space characters only. Tab characters are not allowed. The recommended indentation level is 4 space characters, in line with common clang-format configurations. Code written in an orderly manner, with consistent indentation and alignment across definitions, is much more readable and easier to maintain.

4.2 Comments

Source code without appropriate comments is not usable, even by the same developer after some time from writing.
Comments are used to describe modules, various sections present in a module, functions and body of the functions. Each comment must be exhaustive and follow appropriate rules.
Comments should not be used to exclude portions of code from compilation. To exclude (even temporarily) parts of code, the construct #if 0 … #endif shall be used.

Modules must be commented following the Doxygen standard. The style of comments can be either C (/ *…. * /) Or C ++ (// ....) for the free parts. Some sections of the modules have constrained templates, for which the standard style must be used as described in the following.

4.3 Header file

Each file must start with a standard header. The header uses Doxygen keywords.

```
/*
 *    @file [FileName].h
 *    @date [Date]
 *
 *    @brief [Brief Desc]
 *
 *    @b Description @par
 *          [Description]
 *
 *    @copyright
 * \@Copyright(c) 2021, BAT, All rights reserved. @par
 * No part of this document must be reproduced in any form - including copied,
 * transcribed, printed or by any electronic means - without specific written
 * permission from BAT
 *
 */
```

The fields in green depend on the single file, the other elements are common to all.
In detail, fields have the following meaning:

[FileName]	File name and extension, without the full path.
[Brief Desc]	Brief description of the module.
[Description]	Detailed description of the file / module functionality. In practice, the use of the module will be explained without making specific reference to the individual functions. The description must be able to explain the generic use of the module and any particularities. For specific definitions of the features, we will refer to the comments of the individual methods
[Date]	File creation date. Date in the format “dd/mm/yyyy”.

4.4 Functions

Before each function / method or its prototype, a properly formatted descriptive comment must be inserted. The comment will follow the Doxygen standard in order to be automatically inserted into the documentation. If the function has a prototype in the header file (it is a function visible outside the module) the comment is placed on the prototype. If the function is static (local to the module) the comment must be placed before the function declaration.
The template is the following:

```
/*
 *    @brief      [Description]
 *    @param      [Param Name]      [Param desc.]
 *    @return     [Type]            [Return desc.]
 */
```

The fields in green depend on the single function, the other elements are common to all.
In detail, fields have the following meaning:

[Description]	Detailed description of the function.
[Param name]	Name of the parameter.
[Param desc.]	Parameter description.
[Type]	Type of data returned by the function.
[Return desc.]	Description of data returned by the function. If it is returned void, the description should not be entered.

If the function has several input parameters, a field @param must be entered for each parameter, correctly aligned and indented. If the function has no parameters, the @param field must be replaced in the function comment with:

```
 *    @b No @b Parameters
```

If a parameter is a pointer it should be specified if the pointer is used as input [in], output [out] or input/output [in/out], for example:

```
*     @param[in]  [Param Name]      [Param desc.]
```

4.5 Definitions

Before defining each structure or enum, a comment must always be inserted in Doxygen style. The individual fields of the structure must also be adequately commented.
The comment to the structure will have the following form:

```
/*
 * @brief   [Desc]
 */
```

The [Desc] field simply contains the detailed description of the use of the structure or enum.

In the individual fields, the comment will be inserted on the same line as the definition, appropriately aligned and indented. The comment will be inserted using the Doxygen standard in the form:

```
///< [FieldDesc]
```

In the case of comments concerning macros, it is possible to use both the line form (/// <...) and the prefix form (@brief). The standard used is always that of Doxygen.

4.6 Sections

Each header and source file can be organized into sections. Each section is defined by means of an appropriate comment. The comment is not part of the Doxygen documentation.

```
//=============================================================================
// @@ [Section name]
//=============================================================================
```

The green field contains the name of the section. The symbol "@@" has no special encoding, but can be used as a tag for searches into the document.

4.7 Internal comments on functions

Inside the functions it is useful to insert comments to describe in a way that is independent from the C code what the function is performing. It is not necessary to write the 'translation' of the code, but it is necessary to describe step by step which processes are running. If local variables internal to a function are defined, it is useful to insert a comment next to them to describe their use. Both in-line and block comments are allowed. Block comments must be inserted using the appropriate style:

Correct:

```
/*                      <C Style>
 * .....
 * .....
 */
```

or

```
// .....                <C++ Style>
// .....
// .....
```

or

```
///< .....              <Doxygen Style>
///< .....
```

```
                    ///< .....
```

Wrong:

```
                    /* . . .                    <Multi-line Style>
                     . .
                     . .
                     . . . */
```

Comments, both in-line and in blocks, have the same indentation as the section of code they refer to and they are inserted just before the code.
It is also possible to put short in-line comments on a statement, if needed.

5 Macro

There should be no unexplained references to numeric values in the code. Macro-defined symbolic constants (the #define construct) must be used for numeric values that have specific domain, configuration, or implementation meanings.
Macro definitions, including macro values, shall follow the common project formatting rules for indentation and alignment. The detailed indentation and alignment behaviour shall be controlled by the project clang-format configuration to ensure consistent formatting across the codebase.

Corretto:

```
#define TRIANGLE_BASE          (100)
#define TRIANGLE_HEIGHT        (100)

      TriangleArea      = ((TRIANGLE_BASE * TRIANGLE_HEIGHT) / 2);
```

Errato:

```
      TriangleArea      = ((100 * 100) / 2);
```

If logical or Boolean states are used, the bool type and the constants true and false defined in <stdbool.h> shall be used instead of the numeric values 0 and 1.

5.1 Macro definitions

All macros must be grouped into a well-defined section of the code. Generally macros are inserted in the “LOCAL DEFINITIONS” section of the sources. Macros defined in one module and usable by others are inserted in the header file. Macros used only inside a module must be defined in the appropriate section of the source file.
When defining a macro (or a group of macros), it is necessary to indicate as a prefix of the macro some reference to the module that defined them.
Macros must be defined using uppercase characters separating words with the '_' symbol.
A macro must have only one definition within the same project, even if the definitions are identical.
Brackets must be placed around each parameter or value of the macro. This prevents operator precedence problems when the macro is pre-processed.

Correct:

```
#define MULTIPLY(a,b)          ((a) * (b))
```

Wrong:

```
#define MULTIPLY(a,b)          (a * b)
```

In the correct case, the translation of the macro MULTIPLY (10 + 20, 30) rightly returns result 900.
In the wrong case, the macro is translated as "10 + 20 * 30" and therefore the result is 610 since the operator '* 'takes precedence over the' + 'operator.
Macros must be properly commented according to the directives in the paragraph 4.5.

Multi-line C macros shall be wrapped inside a *do {} while(0)* loop like:

Correct:

```
#define CALL_FUNCS(x)          \
            do {               \
                  func1(x);    \
                  func2(x);    \
            } while (0)
```

If a multiline macro is defined with the *do {} while(0)* approach it will be possible to use the macro in this way:

```
if (<condition>)
  CALL_FUNCS(a);
else
  bar(a);
```

Wrong:

If a multiline macro is defined with the {} block approach as below, the previous usage will rise a compilation error:

```
#define CALL_FUNCS(x)          \
            {                  \
                  func1(x);    \
                  func2(x);    \
            }
```

5.2 Header Hat

A special type of macro is the 'hat' to header files. Each header file must have the appropriate definition in the head to prevent it from being included multiple times in the same file. The format of this section is:

```
#ifndef [HEADER_HAT]
#define [HEADER_HAT]

      // Header body

#endif // [HEADER_HAT]
```

Header hat macro names should avoid leading or trailing double underscores, as these identifiers may conflict with compiler- or implementation-reserved macros. The recommended format is HEADER_FILE_NAME_H, for example CAN_DRIVER_H.

6 Type definitions

When defining a new data type (using the 'typedef' construct) some rules must be followed.
Each new type of data must be commented in a general way and for each single field.
If the data types thus defined by a module are also used by others, they will be inserted in the module header file, if instead they are used only within the module, they will be inserted in the source file.
The reference section is “LOCAL DEFINITIONS” at the end of the macro definitions.
Within the same project, the same data type must not be defined more than once even if the definitions coincide.
To name the data types, a standard similar to that of files will be used: the string '_t' is inserted as a postfix to indicate that it is a user-defined data type, and for the name will be used words separated by '_'.
For example, to create a black box data type the name will be black_box_t.

```
/*
 * @brief [Struct type description]
 */
typedef struct
{
      [type]      [field1];   ///< [field1 comment]
      [type]      [field2];   ///< [field2 comment]
} struct_name_t;
```

7 Variables and constants

The definition of variables and constants follows specific nomenclature and scope rules.
Variables can be local to a function or global. Constants must always be declared global. If there is a real need to declare a local constant to a function, the choice must be justified by adequately commenting on the reason.
The nomenclature of variables and constants is similar to the nomenclature of files. The name of the variable or constant must be explanatory of the function it covers, if made up of several words, they are separated with the symbol '_'.
If it is a pointer, the variable name will be terminated with the postfix '_ptr' or start with prefix 'p_'

Examples of scope reference will be analyzed in the next chapter.

Overall examples:

- Local variable unsigned 8 bit: *uint8_t counter;*
- Global module variable, unsigned 16 bit: *uint16_t g_counter;*
- Static module variable, unsigned 16 bit: *static uint16_t s_counter;*
- function parameter, pointer to a char: *char *name_ptr;* or *char *p_name;*
- constant of a data type defined by user: *const color_t c_color_background*;

7.1 Standard variables datatype

For standard variables datatype stdint and stdbool shall be used.

#include <stdbool.h>	
boolean	bool
#include <stdint.h>	
Unsigned int 8 bit	uint8_t uint_fast8_t
Signed int 8 bit	int8_t int_fast8_t
Unsigned int 16 bit	uint16_t uint_fast16_t
Signed int 16 bit	int16_t int_fast16_t
Unsigned int 32 bit	uint32_t uint_fast32_t
Signed int 32 bit	int32_t int_fast32_t
…	…

7.2 Constants definitions

Constants can be both global to the project and global to the single module, while they do not have to be declared locally in a function. If there is a real need to declare a local constant to a function, the choice must be justified by adequately commenting on the reason.

7.3 Definition of Variables

Variables are divided into two types: local and global. Local variables have no prefix and must have a comment (even in-line) at the time of declaration.

Global variables can be of two families: global to the project and global to the module. In principle, global variables to a project shall not be used, but shall be encouraged the usage of global variables in a module then implementing functions for reading and writing them. However, if there is a need to add global variables to the project, this is not forbidden, but the choice must be illustrated by means of appropriate comments during the declaration phase.

The global variables must all be declared in the “GLOBAL VARIABLES” section of the module source file. If the variables are also global to the project, the definition must be exported via the module header file, again in the "GLOBAL VARIABLES" section.
It is not allowed to define 'extern' type variables within a source file. All global variables will only be accessible by including the appropriate header file.

All global variables have a prefix code which depends on the scope of the variable itself:

g_	Global variable to the project, defined in the source file and exported via the header file.
s_	Module-only global variable, defined in the source file and declared as 'static'.
f_	'Friend' variable. If a module consists of several source files, it is possible to declare global variables only for the module files. To do this, it is necessary to insert an additional header file, which will not be included in other modules, containing all the definitions and exports of the variables common to the module. The variable will be declared in one of the files belonging to the module.

Within the same project, global variables to a modules with the same name shall not be declared. If two modules have a global variable of type 's_' (or 'f_') with the same name, to distinguish them a prefix to the module-dependent name shall be placed.

8 Functions

Defining a function involves declaring both the function itself and the prototype. The positioning of the function and the prototype within the module depends on the scope of the function, whether it is global to the project or only to the module. The nomenclature of the functions follows the rules of the files. The name of the function must be explanatory of the task performed. If the functions belong to a module used by others (for example a library), all the names of the functions will have a prefix depending on the name of the module itself.

Functions that have specific features, shall be named with a postfix that indicates the type of task performed. Examples of such functions are:

- interrupt services routine *(_isr*)
- task entry point (*_task*)
- initialization functions (*_init*)
- functions that enable / disable certain functionalities (*_en / _disable*)

For example, considering a module for the management of a generic SPI device, the functions will have a nomenclature of the type:

Correct:
```
void spi_init(uint32_t bit_rate);
Void spi_send(gps_msg_t message);
void spi_read(gps_msg_t *message_ptr);
```

Wrong:
```
void InitSpi(int32u i32uBaud);
void SendMessage(tMsg tMyMessage);
void SPIReadData(tMsg *tMyMessagePtr);
```

The declaration of each function and prototype must be appropriately commented on the basis of the paragraph 4.4.

8.1 Global functions

The global functions to the project must have the declaration of the body inside the source file in the “PUBLIC FUNCTIONS IMPLEMENTATION” section, while the declaration of the prototype will be inserted in the header file of the module in the “PUBLIC FUNCTION” section.

8.2 Local functions

Local functions to a module will have the body and prototype declaration inside the source file.
The prototype will be declared in the “PRIVATE FUNCTIONS PROTOTYPES” section, while the body in the “PRIVATE FUNCTIONS IMPLEMENTATION” section. The "private" functions to the module must be declared as 'static'.
If the module is made up of several source files, and there are shared functions between the various files of the module (but not usable by other modules), it will be necessary to move the definitions of the prototypes in a special header file. This header file will be included by the module files only and will not be usable by other modules of the project.

Global function declarations shall be defined in the header file of their respective module and shall not be declared static. Each layer’s top-level header shall include the relevant module headers so that users can access the declarations by including the top-level header only.

8.3 Function content

A function must perform a single task or a specific task. In case the function has to perform multiple tasks, the function has to be broken down into multiple functions.
If there are repeated code sections in the project or module, they must be replaced with a single function. Having repeated sections of code, even within the same function, leads to problems during project modifications or maintenance.
A routine must have a number of lines of code necessary to perform a complete, independent and inseparable function. Limitations based on the number of lines of code, or number of pages could affect the design of the function itself. However, a very long function can generally be broken into several sections and then organized into several separate routines.

8.4 Function parameters

If a function needs parameters by value, and if those parameters are not standard types (*stdlib, stdbool*), to improve performance (and avoid copies) it is strongly suggested to pass the pointer to the structure using 'const'.

```
typedef struct
{
      uint32_t    data;
      size_t      size;
      uint32_t    flags;
} data_t;
```

Correct:

```
void foo_perform(const data_t *p_data);
```

incorrect:

```
void foo_perform(data_t data);
```

Furthermore, it shall be avoided to pass pointers to a function with a type different than that declared in the function prototype by forcing the casting.

9 Coding style

Coding style must be consistent across the entire team, regardless of who writes the code. Indentation, alignment, column length, bracket format, and spacing around control statements and operators shall be controlled by the project clang-format configuration. Microsoft style should be used as the base configuration, with project-specific changes applied where required. Where possible, VSCode should be configured to apply clang-format automatically when saving source files.

- Use the project clang-format configuration, based on Microsoft style.

```
int foo(bool is_bar)
{
      if (is_bar)
      {
            bar();
            return 1;
      }
      else
      {
            return 0;
      }
}
```

- Indentation of switches:

```
switch (foo)
{
    case 1:
        a += 1;
        break;

    case 2:
    {
        a += 2;
        break;
    }
}
```

- Insert spaces after the param headers (if, for, while,… ..) and between the operators:

```
if (a == 2)
    b = (c / 3) * 6;
```

- Do not insert multiple statements on the same line.
- Related variable declarations should be grouped together and may be aligned within the same group for readability. Alignment should not be applied across unrelated groups and should not be forced when it causes long lines, comment wrapping, or conflicts with the project clang-format configuration.

```
uint8_t     read_ch     = 10u;
uint32_t    counter    = 100u;
uint32_t    *p_address;


typedef struct
{
      uint8_t     foo;        ///< foo item
      char        dummy;      ///< dummy item
      uint32_t    second;     ///< Second item
} my_struct_t;
```

- If a line needs to be broken, align the parts according to a common logic.

```
a = (a * b) +
    (c * d);

if ((a == b) &&
     (c == d))
```

10 Other prescriptions

- **Jumps**: the use of jumps in the code shall be avoided.
- **Naming**: when naming variables, constants and functions, language or preprocessor keywords shall be avoided.
- **Casting**: in general, the following rules should be observed in relation to casting. In case of specific need to violate them, the reason shall be clearly explained and documented in the code.
  - A cast shall not be performed between a pointer to object type and a pointer to a different object type.
  - A cast shall not be performed between pointer to void and an arithmetic type.
  - A cast shall not be performed between pointer to object and a non-integer arithmetic type.
  - A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.

11 Templates

The standard templates for modules are provided in the following sections.
Both the header file and the source file of each module shall follow the standard.
Fields between '[' and ']' indicate the components dependent on the module itself that must be set each time.

11.1 File header

```
/*
 *    @file [FileName].h
 *    @date [Date]
 *
 *    @brief [Brief Desc]
 *
 *    @b Description @par
 *          [Description]
 *
 *    @copyright
 * \@Copyright(c) 2021, BAT, All rights reserved. @par
 * No part of this document must be reproduced in any form - including copied,
 * transcribed, printed or by any electronic means - without specific written
 * permission from BAT
 *
 */


#ifndef FILE_NAME_H
#define FILE_NAME_H

#include <[Files.h]>

//=========================================================================================
// @@ PUBLIC DEFINITIONS
//=========================================================================================

//=========================================================================================
// @@ GLOBAL VARIABLES
//=========================================================================================

//=========================================================================================
// @@ CONSTANTS
//=========================================================================================

//=========================================================================================
// @@ PUBLIC FUNCTION
//=========================================================================================

#endif //FILE_NAME_H
```

Definitions of the various sections must be present even if the section is empty. If there are more files to include, they will all be inserted under the "hat" of the header.

11.2 File source

```
/*
 *    @file [FileName].c
 *    @date [Date]
 *
 *    @brief [Brief Desc]
 *
 *    @b Description @par
 *          [Description]
 *
 *    @copyright
 * \@Copyright(c) 2021, BAT, All rights reserved. @par
 * No part of this document must be reproduced in any form - including copied,
 * transcribed, printed or by any electronic means - without specific written
 * permission from BAT
 *
 */

#include "[FileName].h"

#include <[Files.h]>

//=============================================================================
// @@ LOCAL DEFINITIONS
//=============================================================================

//=============================================================================
// @@ GLOBAL VARIABLES
//=============================================================================

//=============================================================================
//    @@    CONSTANTS
//=============================================================================

//=============================================================================
// @@ PRIVATE FUNCTIONS PROTOTYPES
//=============================================================================


//=============================================================================
// @@ PUBLIC FUNCTIONS IMPLEMENTATION
//=============================================================================


//=============================================================================
// @@ PRIVATE FUNCTIONS IMPLEMENTATION
//=============================================================================
```

Definitions of the various sections must be present even if the section is empty. If there are more files to include, they will all be inserted under the "hat" of the header.
The header part of the source file and the header will be the same.

12 Memory & Resource Management

This section defines rules for allocating, releasing, and bounding memory and other system resources (files, handles, streams). These rules exist because memory and resource errors : leaks, double-frees, use-after-free, and out-of-bounds access are among the most common causes of field failures in embedded firmware.

12.1 Dynamic Memory Allocation

Dynamic (heap) memory allocation (malloc/free family) should not be used in BAT firmware unless explicitly approved for a specific subsystem. Static or stack allocation is preferred, since heap fragmentation and allocation failure are difficult to test exhaustively on constrained targets.

Where dynamic allocation is unavoidable and approved, every successful allocation must have exactly one corresponding release on every code path, and the pointer must not be used afterward.

Correct:

```
static uint8_t s_rx_buffer[RX_BUFFER_SIZE];/* static allocation, size known at compile
time*/

void rx_buffer_init(void)

{

 memset(s_rx_buffer, 0, sizeof(s_rx_buffer));

}
```

Wrong:

```
uint8_t *buf = malloc(RX_BUFFER_SIZE);   /* heap allocation not approved for this subsystem
*/

process(buf);/* buf is never freed -- and its success is never checked */
```

Covers SonarQube rules #45, #53, #192, #24, #25, #30, #35 (see SonarQube rule mapping table in section §20).

12.2 Resource Closing

Any resource obtained through an explicit open/acquire call (file handle, stream, peripheral lock) must be closed/released on every exit path of the function that owns it, including error paths.

Correct:

```
FILE *fp = fopen(path, "r");

if (fp == NULL) {

return STATUS_ERR_OPEN;

}

status_t st = read_config(fp);

fclose(fp);           /* closed on the normal path ... */

return st;
```

Wrong:

```
FILE *fp = fopen(path, "r");

if (fp == NULL) {

    return STATUS_ERR_OPEN;

}

if (read_config(fp) != STATUS_OK) {

return STATUS_ERR_CONFIG;   /* fp leaked on this path */

}

fclose(fp);
```

Covers SonarQube rules #23, #34 (see SonarQube rule mapping table in section §20).

12.3 Buffer, Array and String Bounds

All buffer, array, and string operations must have an explicit, verifiable bound. Functions from <string.h> that do not take a length argument (strcpy, strcat, strlen, sprintf, gets) must not be used; use the bounded equivalents (strncpy, strncat, snprintf) and always verify the destination size.

Correct:

```
char name[NAME_MAX_LEN];

int n = snprintf(name, sizeof(name), "%s", src);

if (n < 0 || (size_t)n >= sizeof(name)) {

   return STATUS_ERR_TRUNCATED;

}
```

Wrong:

```
char name[NAME_MAX_LEN];

strcpy(name, src);   /* no bound -- overflows if src is longer than NAME_MAX_LEN */
```

Covers SonarQube rules #29, #140, #194, #216, #252, #384, #81, #410-#415, #402, #251, #295, #17, #210 ((see SonarQube rule mapping table in section §20).

12.4 Format Strings

Format strings passed to printf/scanf-family functions must be compile-time string literals, never data derived from external input, and every %s field width must be bounded.

Correct:

```
printf("Value: %d\n", value);

scanf("%9s", name_buf);         /* field width bounds the write */
```

Wrong:

```
printf(user_supplied_string);   /* format string from untrusted input */

scanf("%s", name_buf);          /* unbounded write */
```

Covers SonarQube rules #4, #31, #183, #263, #352, #163, #164 (see SonarQube rule mapping table in section §20).

13 Security Practices

This section defines rules for cryptography, secrets handling, banned/insecure library functions, and filesystem/environment security. These rules exist because BAT firmware increasingly handles network connectivity, provisioning secrets, and OTA updates, and insecure defaults in these areas create field-exploitable vulnerabilities rather than just functional bugs.

13.1 Cryptography and TLS

Cipher algorithms, cryptographic keys, and encryption modes must be robust (no weak/legacy algorithms). TLS/SSL connections must verify both the server certificate and hostname; connections must not silently accept unverified

certificates. Weak hashing algorithms (e.g. MD5, SHA-1 for security purposes) and pseudorandom number generators must not be used for anything security-sensitive.

Correct:

```
mbedtls_ssl_conf_authmode(&conf, MBEDTLS_SSL_VERIFY_REQUIRED);

mbedtls_ssl_set_hostname(&ssl, expected_hostname);

/* connection fails closed if the cert or hostname don't match */
```

Wrong:

```
mbedtls_ssl_conf_authmode(&conf, MBEDTLS_SSL_VERIFY_NONE);

/* accepts any certificate -- defeats the purpose of TLS */
```

Covers SonarQube rules #6, #7, #8, #10, #11, #15, #418, #420.

13.2 Secrets and Credentials

Passwords, API keys, and other secrets must never be hard-coded in source. Temporary file name templates must use at least six trailing 'X' characters. File permissions on any created resource must follow least-privilege, never world-writable.

Correct:

```
const char *api_key = provisioning_get_secret(SECRET_API_KEY);

/* secret is read from secure storage at runtime */
```

Wrong:

```
static const char *api_key = "a1b2c3d4e5f6";  /* hard-coded secret in source */
```

Covers SonarQube rules #406, #407, #168, #409.

13.3 Banned and Insecure Library Functions

The following standard-library functions must not be used, since they are either unsafe by design or invoke a command shell: system(), abort(), exit(), getenv(), atof/atoi/atol (no error detection), bsearch/qsort (from <stdlib.h>, prefer project-vetted equivalents), and obsolete/unauthorized POSIX functions. Use validated, bounds-checked alternatives instead (e.g. strtol with error checking instead of atoi).

Correct:

```
char *endptr;

long val = strtol(str, &endptr, 10);

if (endptr == str || *endptr != '\0') {

    return STATUS_ERR_PARSE;

}
```

Wrong:

```
int val = atoi(str);   /* no way to detect a parse failure */
```

Covers SonarQube rules #12, #9, #43, #44, #112, #211, #286, #319.

13.4 Filesystem and Environment Security

File access must not introduce time-of-check-to-time-of-use (TOCTOU) races. The return value of directory-changing calls (chroot, chdir) must always be checked. Archive expansion must control resource consumption (zip-bomb protection). Hardcoded IP addresses, clear-text protocols, and publicly writable directories are flagged as security-sensitive and require explicit justification if used.

Correct:

```
if (chdir(safe_path) != 0) {
   return STATUS_ERR_CHDIR;
}
/* proceed only after confirming success */
```

Wrong:

```
chdir(safe_path);   /* return value ignored */
do_work();          /* may be running in the wrong directory */
```

Covers SonarQube rules #3, #5, #16, #403, #404, #405, #416, #417, #419.

13.5 Sensitive Data Handling

Sensitive data (keys, passwords, tokens) held in memory must be wiped using a function the compiler cannot optimize away (e.g. a project-provided secure_zero() built on a volatile pointer), not a plain memset(), which compilers are permitted to eliminate as a dead store.

Correct:

```
secure_zero(key_buf, sizeof(key_buf));  /* volatile-based, not optimized out */
```

Wrong:

```
memset(key_buf, 0, sizeof(key_buf));
/* compiler may remove this if key_buf is not read again -- key stays in memory */
```

Covers SonarQube rule #1.

14 Pointer & Null Safety

This section defines rules for pointer lifetime, null-pointer safety, pointer arithmetic, and casting. Pointer misuse is a leading cause of memory corruption and hard-to-reproduce field failures, and is not addressed by the naming/formatting rules elsewhere in this standard.

14.1 Returning or Retaining Pointers to Local Storage

A function must never return a pointer or reference to a local variable with automatic (stack) storage duration. Similarly, the address of a local object must not be assigned to another object, or stored in a struct/global, that can outlive it.

Correct:

```
status_t get_name(char *out, size_t out_len){
    /* caller owns the buffer -- no dangling pointer possible */
```

```
    return snprintf(out, out_len, "%s", s_name) < (int)out_len
  ? STATUS_OK : STATUS_ERR_TRUNCATED;
}
```

Wrong:

```
const char *get_name(void){
    char local_buf[32];
    snprintf(local_buf, sizeof(local_buf), "%s", s_name);
    return local_buf;   /* dangling pointer once the function returns */
}
```

Covers SonarQube rules #22, #37, #50, #13, #66.

14.2 Null Pointer Dereference

Any pointer that can legitimately be NULL (return value of an allocator, an optional parameter, a lookup function) must be checked before dereference.

Correct:

```
node_t *n = list_find(list, key);
if (n != NULL) {
    process(n);
}
```

Wrong:

```
node_t *n = list_find(list, key);
process(n);   /* list_find can return NULL on a miss */
```

Covers SonarQube rules #57, #261.

14.3 Pointer Arithmetic and Casting

Pointer arithmetic must stay within the bounds of the same array/object; subtraction and relational comparisons between pointers are only valid within the same array. Pointers must not be cast to/from integral types other than uintptr_t/intptr_t, cast between function-pointer and other types, or converted outside a safe, well-defined subset. sizeof() must not be applied directly to a pointer when the intent is the size of the pointed-to object.

Correct:

```
for (size_t i = 0; i < count; i++) {
    process(&array[i]);   /* stays within array bounds */
}
```

Wrong:

```
int *p = (int *)0x20001000;   /* integer cast to pointer without a defined reason */
size_t sz = sizeof(p);        /* size of the pointer, not the pointee -- likely a bug */
```

Covers SonarQube rules #60, #61, #63, #64, #71, #72, #79, #84, #91, #101, #103, #113, #131, #152, #170, #179.

14.4 Pointer Indirection Depth

Object declarations should contain no more than two levels of pointer indirection (e.g. int **, not int ***). Deeper indirection is difficult to reason about and usually signals a design that should use a struct or an index instead.

Correct:

```
typedef struct { uint8_t *data; size_t len; } buffer_t;

void process(buffer_t **buffers, size_t count);   /* 2 levels */
```

Wrong:

```
void process(uint8_t ***data, size_t count);   /* 3 levels -- hard to reason about
ownership */
```

Covers SonarQube rules #207, #217.

15 Error Handling & Return Values

This section defines rules for checking return values, initializing variables before use, and handling program-terminating and non-local control transfers. Silently ignored errors and uninitialized reads are a common source of field failures that pass code review because the code 'looks' correct.

15.1 Return Value Checking

Every function with a non-void return type must return a value on all paths, and callers must check return values that indicate success/failure -- especially for functions marked 'nodiscard' and for setuid-family calls. Do not call a function only for its side effect if it can fail.

Correct:

```
if (spi_write(handle, buf, len) != STATUS_OK) {

    return STATUS_ERR_SPI;

}
```

Wrong:

```
spi_write(handle, buf, len);   /* return value ignored -- failure goes unnoticed */
```

Covers SonarQube rules #48, #54, #215, #292, #391, #127.

15.2 Preconditions and Initialization

A function call must not violate the callee's documented preconditions. Every variable must be initialized before its value is read; do not rely on an object's value being set on a path that may not execute.

Correct:

```
uint32_t retries = 0;   /* initialized at declaration */
while (retries < MAX_RETRIES && !done) {
    retries++;
}
```

Wrong:

```
uint32_t retries;
if (needs_retry) {
    retries = 0;
}
while (retries < MAX_RETRIES) {
 /* retries may be read uninitialized */
}
```

Covers SonarQube rules #21, #117, #123, #336.

15.3 Program Termination and Non-local Jumps

Program-terminating functions (abort, exit) and non-local jumps (setjmp/longjmp, <signal.h>, <csignal>, <csetjmp>) must not be used in BAT firmware -- they bypass normal cleanup and are a common source of resource leaks and undefined behaviour on embedded targets. Functions that never return must be declared with the 'noreturn' attribute so the compiler and reviewers can rely on that contract.

Correct:

```
_Noreturn void fatal_error(status_t reason){
    log_fatal(reason);
    watchdog_force_reset();
    for (;;) { }   /* never returns, and is declared as such */
}
```

Wrong:

```
void fatal_error(status_t reason){
    log_fatal(reason);
    exit(1);   /* bypasses cleanup handlers; not appropriate on embedded targets */
}
```

Covers SonarQube rules #27, #42, #62, #171, #224, #303, #355.

15.4 File Stream Positioning

When a file stream is used for both reading and writing, a positioning operation (fseek, fflush, or equivalent) must separate the read and write operations; interleaving reads and writes without repositioning is undefined behaviour in the C standard.

Correct:

```
fread(buf, 1, len, fp);
fseek(fp, 0, SEEK_CUR);   /* explicit positioning operation */
fwrite(buf, 1, len, fp);
```

Wrong:

```
fread(buf, 1, len, fp);
fwrite(buf, 1, len, fp);   /* no positioning operation between read and write */
```

Covers SonarQube rule #32.

16 Standard Compliance / Undefined Behaviour

This section defines rules that keep code within well-defined C/C++ standard behaviour: avoiding side effects in places the standard leaves unsequenced, avoiding duplicate definitions, and never relying on undefined or unspecified behaviour. Code that violates these rules may work correctly on one compiler/optimization level and fail silently on another -- exactly the kind of bug that is expensive to find in the field.

16.1 Side Effects in Unsequenced Expressions

The operands of &&, ||, and sizeof() must not contain side effects (assignments, increments, function calls with side effects). The order of evaluation of function arguments must not be relied upon.

Correct:

```
int idx = get_next_index();
if (idx >= 0 && array[idx] != 0) {   /* no side effect inside the && operands */
...
}
```

Wrong:

```
if (i++ < limit && process(i)) {   /* side effect (i++) inside a logical operand */
    ...}
```

Covers SonarQube rules #14, #89, #157, #313, #52.

16.2 One-Definition Rule and Duplicate Declarations

Every entity (function, object, type) must be defined exactly once across the whole program; multiple declarations of the same variable or function must agree in type.

Correct:

```
/* header */

extern uint32_t g_tick_count;  /* single .c file */

uint32_t g_tick_count = 0;
```

Wrong:

```
/* module_a.c */

uint32_t g_tick_count = 0;

/* module_b.c - redefinition,violates one-definition rule */

uint32_t g_tick_count = 0;
```

Covers SonarQube rules #36, #38, #49.

16.3 Undefined and Unspecified Behaviour

Code must not rely on undefined or critical unspecified behaviour: no division by a possible-zero denominator without a guard, no copying between overlapping objects with a non-overlap-safe function (e.g. memcpy), and no equality comparison between floating-point values.

Correct:

```
if (denominator != 0) {

    result = numerator / denominator;

}...

if (fabsf(a - b) < EPSILON) {

 /* tolerance-based float comparison */

}
```

Wrong:

```
result = numerator / denominator;   /* denominator not checked for zero */

...

if (a == b) {

/* direct float equality is undefined-ish in practice */

}
```

Covers SonarQube rules #39, #51, #58, #67, #90, #134.

17 Preprocessor Safety

This section defines rules for #include hygiene, conditional-compilation directives, and #pragma usage. Preprocessor misuse is invisible in the compiled binary's logic but can silently include the wrong file, evaluate an undefined macro as 0, or make the build non-portable across toolchains.

17.1 #include Hygiene

#include directives must be followed by a well-formed <filename> or "filename" sequence, must not contain non-standard characters, must only be preceded by other preprocessor directives or comments, and must not rely on a non-portable search-path strategy.

Correct:

```
#include "drivers/spi_driver.h"   /* project-relative, portable path */
```

Wrong:

```
#include "../../../drivers/spi_driver.h"   /* relies on a non-portable relative path */
```

Covers SonarQube rules #68, #100, #110, #111, #228, #229, #241, #274, #322.

17.2 Conditional Compilation Directives

Every line beginning with '#' must be a valid, recognized preprocessing directive. Identifiers used in the controlling expression of #if/#elif must be defined before evaluation (avoid relying on an undefined macro silently evaluating to 0). #else/#elif/#endif must reside in the same file as their matching #if/#ifdef/#ifndef. Trigraphs, digraphs, and non-standard forms of the 'defined' operator must not be used.

Correct:

```
#if defined(FEATURE_WIFI) && (FEATURE_WIFI == 1)

  #include "wifi_driver.h"

#endif
```

Wrong:

```
#if FEATURE_WIFI == 1  /* FEATURE_WIFI may not be defined at all silently evaluates to 0*/
```

```
  #include "wifi_driver.h"
#endif
```

Covers SonarQube rules #74, #78, #94, #144, #151, #159, #182, #218, #227, #240.

17.3 #pragma Usage

Every use of #pragma must be documented with a comment explaining why it is needed (compiler-specific behaviour is otherwise invisible to reviewers). #pragma pack must be used correctly (paired push/pop). #include_next, #pragma warning(default:...), and the _Pragma operator must not be used.

Correct:

```
#pragma pack(push, 1)   /* required: this struct is DMA'd byte-for-byte to the radio */
typedef struct {
 uint8_t type; uint16_t len;
}
packet_hdr_t;
#pragma pack(pop)
```

Wrong:

```
#pragma pack(1)   /* undocumented, and never restored -- affects every struct after it */
```

Covers SonarQube rules #69, #248, #323, #381.

18 Concurrency & Thread Safety

This section defines rules for mutex lifecycle management and reentrancy in multi-threaded/RTOS code. Only 5 SonarQube rules fall into this category today, but each is Blocker or High severity -- concurrency bugs (deadlock, double-unlock, non-reentrant calls from an ISR or a second task) are among the hardest defects to reproduce and debug on a running system.

18.1 Mutex Lifecycle

Every mutex must be initialized before first use and destroyed when no longer needed. Mutexes must be unlocked in the reverse order they were locked (to avoid deadlock), and must never be locked when already locked by the same context, or unlocked when already unlocked.

Correct:

```
pthread_mutex_lock(&mutex_a);

pthread_mutex_lock(&mutex_b);/* ... critical section ... */

pthread_mutex_unlock(&mutex_b);   /* unlocked in reverse order */

pthread_mutex_unlock(&mutex_a);
```

Wrong:

```
pthread_mutex_lock(&mutex_a);

pthread_mutex_lock(&mutex_b);/* ... critical section ... */

pthread_mutex_unlock(&mutex_a);   /* unlocked out of order -- deadlock risk elsewhere */

pthread_mutex_unlock(&mutex_b);
```

Covers SonarQube rules #18, #19, #20.

18.2 Reentrancy and Blocking Calls

Non-reentrant POSIX/libc functions (e.g. strtok, localtime, non-reentrant rand) must be replaced with their reentrant equivalents (strtok_r, localtime_r) when called from more than one task or from an ISR context. Blocking calls must not be made inside a critical section (while holding a mutex), since this risks priority inversion and blocking every other task waiting on that lock.

Correct:

```
char *saveptr;

char *tok = strtok_r(input, ",", &saveptr);   /* reentrant, safe across tasks */
```

Wrong:

```
char *tok = strtok(input, ",");
```

```
/* uses hidden internal static state -- not thread-safe */
```

Covers SonarQube rules #148, #184.

19 Arithmetic & Type Conversion Safety

This is the largest of the new-section topics (44 rules). It covers signed/unsigned mixing, integer overflow/wraparound, implicit conversions that lose precision, and operator misuse. These issues are subtle because the code compiles and often 'works' on the test bench, then misbehaves at a boundary value in the field.

19.1 Signed / Unsigned Mixing

Signed and unsigned types must not be mixed in the same expression without an explicit, intentional cast. Bitwise and shift operators must not be applied to signed operands. The sign of an unsigned variable must never be tested (it cannot be negative, so such a test is either dead code or a logic error), and unary minus must not be applied to an unsigned expression.

Correct:

```
uint32_t count = get_count();

if (count == 0) {   /* explicit, well-defined comparison */

    ...

}
```

Wrong:

```
uint32_t count = get_count();

if ((int32_t)count < 0) {   /* testing the sign of what was an unsigned value */

 ...}
```

Covers SonarQube rules #82, #114, #116, #118, #223, #373, #328.

19.2 Overflow and Wraparound

Integral and unsigned arithmetic operations must not be allowed to overflow or silently wrap, even when operands are compile-time constants. Values must not be uselessly incremented (dead stores) or incremented purely to set a boolean to true.

Correct:

```
if (count < UINT32_MAX) {

    count++;   /* guarded against wraparound */

}
```

Wrong:

```
count++;   /* wraps silently to 0 once count == UINT32_MAX */
```

Covers SonarQube rules #80, #93, #122, #276.

19.3 Implicit Conversion and Casting Precision

Implicit casts must not lower precision (e.g. assigning a 32-bit value into a 16-bit variable without an explicit, reviewed cast). A pointer type must not be cast to an integral type other than uintptr_t/intptr_t. The result of a cast to a narrower type must have the same signedness as the source expression.

Correct:

```
uint16_t narrow_id = (uint16_t)(wide_id & 0xFFFFu);   /* explicit, bounded cast */
```

Wrong:

```
uint16_t narrow_id = wide_id;

 /* implicit truncation from uint32_t, no cast, no bounds check */
```

Covers SonarQube rules #273, #243, #250, #306.

19.4 Operator Usage and Precedence

The comma operator should not be used. Increment/decrement operators must not be mixed with other operators in the same expression. Reliance on operator precedence should be limited -- use parentheses to make intent explicit. Non-existent operators (e.g. '=+' where '+=' was intended) must not appear. Boolean expressions must not be used as operands to arithmetic operators, and vice versa.

Correct:

```
total = total + delta;   /* clear, unambiguous, prefix form for increments */

++index;
```

Wrong:

```
total =+ delta;   /* likely meant += -- silently parsed as total = (+delta) */

x = a++ + ++a;    /* mixed increment/decrement in one expression -- unsequenced */
```

Covers SonarQube rules #99, #203, #230, #257, #275, #278, #302, #325, #382, #387, #92, #83.

19.5 Literals, Escape Sequences and String Concatenation

Only escape sequences defined in the ISO C standard should be used in character/string literals; control characters must not appear directly in literals. String literals with different encoding prefixes must not be concatenated (implicitly or otherwise). Multicharacter literals ('ab') must not be used.

Correct:

```
const char *msg = "Line 1\nLine 2";   /* standard escape sequence */
```

Wrong:

```
const char *msg = "Line 1\qLine 2";   /* \\q is not a defined ISO C escape sequence */
```

Covers SonarQube rules #102, #125, #188, #108, #109, #375, #365.

19.6 Enumerations and Loop Counters

Values of different enum types must not be compared to each other. Enum values must only be used as operands to a restricted operator set. Within an enumerator list, implicitly-specified values must be unique and must not collide with explicitly-specified ones. 'for' loop counters must not have floating-point type, and switch conditions must not have essentially boolean type.

Correct:

```
for (uint32_t i = 0; i < count; i++) {   /* integer loop counter */

    ...

}
```

Wrong:

```
for (float f = 0.0f; f < 10.0f; f += 0.1f) {
```

```
  /* floating-point loop counter -- imprecise termination */
...
}
```

Covers SonarQube rules #121, #232, #233, #321, #126, #377, #379, #2.

20 Appendix

This coding standard incorporates relevant SonarQube rules to support code quality, reliability, maintainability, and security. The rule mapping enables compliance checks and consistent application of coding requirements.

1	"memset" should not be used to delete sensitive data
2	POSIX functions should not be called with arguments that trigger buffer overflows
3	XML parsers should not be vulnerable to XXE attacks
4	"scanf()" and "fscanf()" format strings should specify a field width for the "%s" string placeholder
5	Accessing files should not introduce TOCTOU vulnerabilities
6	Cipher algorithms should be robust
7	Cryptographic keys should be robust
8	Encryption algorithms should be used with secure mode and padding scheme
9	Insecure functions should not be used
10	Server certificates should be verified during SSL/TLS connections
11	Server hostnames should be verified during SSL/TLS connections
12	The library function "system" from "<cstdlib>" shall not be used
13	The pointer returned by the C++ Standard Library functions "asctime", "ctime", "gmtime", "localtime", "localeconv", "getenv", "setlocale" or "strerror" must not be used following a subsequent call to the same function
14	The right-hand operand of a logical "&&" or "||" operator should not contain "persistent side effects"
15	Weak SSL/TLS protocols should not be used
16	Account validity should be verified when authenticating users with PAM
17	"memcmp" should only be called with pointers to trivially copyable types with no padding
18	"pthread_mutex_t" should be properly initialized and destroyed
19	"pthread_mutex_t" should be unlocked in the reverse order they were locked
20	"pthread_mutex_t" should not be locked when already locked, or unlocked when already unlocked
21	A function call shall not violate the function's preconditions
22	A function must not return a reference or a pointer to a local variable with automatic storage duration
23	Closed resources should not be accessed
24	Dynamically allocated memory should be released
25	Freed memory should not be used
26	Function-like macros should not be invoked without all of their arguments
27	Functions with "noreturn" attribute should not return
28	Macro arguments should not contain preprocessing directives
29	Memory access should be explicitly bounded to prevent buffer overflows
30	Memory locations should not be released more than once
31	Printf-style format strings should not lead to unexpected behavior at runtime
32	Reads and writes on the same file stream shall be separated by a positioning operation

33	Recursion should not be infinite
34	Resources should be closed
35	Stack allocated memory and non-owned memory should not be freed
36	The "one-definition rule" shall not be violated
37	The address of an automatic object should not be assigned to another object that may persist after the first object has ceased to exist
38	The source code used to implement an "entity" shall appear only once
39	There shall be no occurrence of "undefined" or "critical unspecified behaviour"
40	Tokens that look like a preprocessing directive shall not occur within a macro argument
41	Variables should not be accessed outside of their scope
42	"<signal.h>" should not be used
43	"abort", "exit", "getenv" and "system" from <stdlib.h> should not be used
44	"atof", "atoi" and "atol" from <stdlib.h> should not be used
45	"Dynamic memory" should not be used
46	"volatile" should not be used to qualify objects for which the meaning is not defined
47	A "header file" shall not contain definitions of functions or objects that are non-inline and have external linkage
48	A function with non-"void" return type shall return a value on all paths
49	All "declarations" of a variable or function shall have the same type
50	An assignment operator shall not assign the address of an object with automatic storage duration to an object with a greater lifetime
51	An object or subobject must not be copied to an overlapping object
52	Arguments evaluation order should not be relied on
53	Dynamic heap memory allocation should not be used
54	Function exit paths should have appropriate return values
55	Line-splicing shall not be used in "//" comments
56	Line-splicing should not be used in "//" comments
57	Null pointers should not be dereferenced
58	Operations on a memory location shall be sequenced appropriately
59	Parameter values should be appropriate
60	Pointer arithmetic shall not form an invalid pointer
61	Pointers should not be cast to integral types
62	Program-terminating functions should not be used
63	Relational and subtraction operators should not be used with pointers to different arrays
64	Subtraction between pointers shall only be applied to pointers that address elements of the same array
65	The "volatile" qualifier shall be used appropriately
66	The pointers returned by the C++ Standard Library functions "localeconv", "getenv", "setlocale" or "strerror" must only be used as if they have pointer to const-qualified type
67	Zero should not be a possible denominator
68	"#include" directives should be followed by either <filename> or "filename" sequences
69	"#pragma pack" should be used correctly
70	"case" ranges should not be empty
71	"sizeof" should not be called on pointers
72	"sizeof(sizeof(...))" should not be used
73	A function shall not contain "unreachable" statements
74	A line whose first token is "#" shall be a valid preprocessing directive

75	A named bit-field with "signed integer type" shall not have a length of one bit
76	All branches in a conditional structure should not have exactly the same implementation
77	All code should be reachable
78	All identifiers used in the controlling expression of "#if" or "#elif" preprocessing directives shall be defined prior to evaluation
79	An object with integral, enumerated, or pointer to "void" type shall not be cast to a pointer type
80	An unsigned arithmetic operation with constant operands should not wrap
81	Array values should not be replaced unconditionally
82	Bitwise operators should not be applied to signed operands
83	Boolean operations should not have numeric operands, and vice versa
84	Casts shall not be performed between a pointer to function and any other type
85	Conditionally executed code should be reachable
86	Controlling expressions should not be invariant
87	Declaration specifiers should not be redundant
88	Enums should be consistent with the bit fields they initialize
89	Evaluation of the operand to the sizeof operator shall not contain side effects
90	Floating point numbers should not be tested for equality
91	Function pointers should not be converted to any other type
92	Identical expressions should not be used on both sides of a binary operator
93	Integral operations should not overflow
94	Lines starting with "#" should contain valid preprocessing directives
95	Local variables shall not have static storage duration
96	Loops with at most one iteration should be refactored
97	Macros used in preprocessor directives should be defined before use
98	Non-empty statements should change control flow or have at least one side-effect
99	Non-existent operators like "=+" should not be used
100	Non-standard characters should not occur in header file names in "#include" directives
101	Objects with integer type should not be converted to objects with pointer type
102	Only escape sequences defined in the ISO C standard should be used
103	Pointer conversions should be restricted to a safe subset
104	Precautions shall be taken in order to prevent the contents of a "header file" being included more than once
105	Related "if/else if" statements should not have the same condition
106	Results of ~ and << operations on operands of underlying types unsigned char and unsigned short should immediately be cast to the operand's underlying type
107	Single-bit named bit fields should not be of a signed type
108	String literals with different encoding prefixes shall not be concatenated
109	String literals with different prefixes should not be concatenated
110	The "#include" directive shall be followed by either a "<filename>" or ""filename"" sequence
111	The "'" or "'" or "\" characters and the "/*" or "//" character sequences shall not occur in a "header file" name
112	The "<stdlib.h>" functions "bsearch" and "qsort" should not be used
113	The built-in relational operators ">", ">=", "<" and "<=" shall not be applied to objects of pointer type, except where they point to elements of the same array
114	The built-in unary "-" operator should not be applied to an expression of unsigned type
115	The number of arguments passed to a function should match the number of parameters

116	The operands of "bitwise operators" and "shift operators" shall be appropriate
117	The value of an object must not be read before it has been set
118	Unary minus should not be applied to an unsigned expression
119	Unary prefix operators should not be repeated
120	User-defined types should not be passed as variadic arguments
121	Values of different "enum" types should not be compared
122	Values should not be uselessly incremented
123	Variables should be initialized before use
124	Variables should not be self-assigned
125	Within character literals and non raw-string literals, "\" shall only be used to form a defined escape sequence or universal character name
126	"for" loop counters should not have essentially floating type
127	"nonnull" parameters and return values of "returns_nonnull" functions should not be null
128	Function parameters' initial values should not be ignored
129	Line continuation characters '\' should not be followed by trailing whitespace
130	Parentheses should be used to make the meaning of an expression appropriately explicit
131	The "sizeof" and "alignof" operator should not be used with operands of a "void" type
132	The names of the "standard signed integer types" and "standard unsigned integer types" should not be used
133	The structure of a "switch" statement shall be appropriate
134	Floating-point arithmetic should be used appropriately
135	"case" ranges should cover multiple values
136	"goto" should jump to labels declared later in the same function
137	"goto" statements should not be used to jump into blocks
138	"switch" statements should not contain non-case labels
139	A "goto" statement shall reference a label in a surrounding block
140	Array indices should be placed between brackets
141	Bit fields should not be used
142	Bit-fields should not be declared
143	Control should not be transferred into a complex logic block using a "goto" or a "switch" statement
144	Digraphs should not be used
145	In the definition of a function-like macro, each instance of a parameter shall be enclosed in parentheses, unless it is used as the operand of # or ##
146	Keywords introduced in later specifications should not be used as identifiers
147	Names of well-known C standard library macros and functions should not be used as identifiers
148	Non-reentrant POSIX functions should be replaced with their reentrant versions
149	Octal constants shall not be used
150	Octal values should not be used
151	Only standard forms of the "defined" directive should be used
152	Redundant pointer operator sequences should be removed
153	Reserved identifiers should not be defined or declared
154	Switch cases should end with an unconditional "break" statement
155	Switch labels should not be nested inside non-switch blocks
156	The "goto" statement shall jump to a label declared later in the function body
157	The right-hand operands of && and || should not contain side effects
158	Track lack of copyright and license headers

159	Trigraphs should not be used
160	User-defined identifiers shall have an appropriate form
161	"#undef" should be used with caution
162	"#undef" should only be used for macros defined previously in the same file
163	"<stdio.h>" should not be used in production code
164	"<time.h>" should not be used
165	"default" clauses should be first or last
166	"errno" should not be used
167	"if ... else if" constructs should end with "else" clauses
168	"mktemp" family of functions templates should have at least six trailing "X"s
169	"offsetof" macro should not be used
170	"restrict" should not be used
171	"setjmp" and "longjmp" should not be used
172	"static" should not be used for the size of an array parameter
173	"switch" statements should have "default" clauses
174	"switch" statements should not be nested
175	"typedef" should be used for function pointers
176	A cast shall not remove any "const" or "volatile" qualification from the type accessed via a pointer or by reference
177	A cast shall not remove any const or volatile qualification from the type of a pointer or reference
178	A conditionally executed single line should be denoted by indentation
179	A conversion from function type to pointer-to-function type shall only occur in appropriate contexts
180	A macro parameter immediately following a "#" operator shall not be immediately followed by a "##" operator
181	A variable declared in an "inner scope" shall not hide a variable declared in an "outer scope"
182	All "#else", "#elif" and "#endif" preprocessor directives shall reside in the same file as the "#if", "#ifdef" or "#ifndef" directive to which they are related
183	Argument of "printf" should be a format string
184	Blocking functions should not be called inside critical sections
185	Cognitive Complexity of functions should not be too high
186	Conditionals should start on new lines
187	Constants of unsigned type should have a "U" suffix
188	Control characters should not be used in literals
189	Control flow statements "if", "for", "while", "switch" and "try" should not be nested too deeply
190	Control structures should use curly braces
191	Cyclomatic Complexity of functions should not be too high
192	Dynamic memory shall be managed automatically
193	Expressions should not be too complex
194	Flexible array members should not be declared
195	Function names should be used either as a call with a parameter list or with the "&" operator
196	Function-like macros shall not be defined
197	Function-like macros should not be used
198	Functions shall not call themselves, either directly or indirectly
199	Functions should be declared explicitly
200	Functions should not be defined with a variable number of arguments
201	Functions without parameters should be declared with parameter type "void"

202	Header guards should be followed by a matching "#define" macro
203	Logical operators should not be confused with bitwise operators
204	Macros should not be #define'd or #undef'd within a block
205	Methods should not be empty
206	Object and function types should be explicitly stated in their declarations and definitions
207	Object declarations should contain no more than 2 levels of pointer indirection
208	Octal and hexadecimal escape sequences should be terminated
209	Octal escape sequences, hexadecimal escape sequences and universal character names shall be terminated
210	Only valid arguments should be passed to stream functions
211	Only valid arguments should be passed to UNIX/POSIX functions
212	Pre-defined macros should not be defined, redefined or undefined
213	Preprocessor directives should not be indented
214	Recursion should not be used
215	Return value of "setuid" family of functions should always be checked
216	Size of variable length arrays should be greater than zero
217	The "declaration" of an object should contain no more than two levels of pointer indirection
218	The "defined" preprocessor operator shall be used appropriately
219	The body of an "iteration-statement" or a "selection-statement" shall be a "compound-statement"
220	The features of "<cstdarg>" shall not be used
221	The literal value zero shall be the only value assigned to "errno"
222	The macro "offsetof" shall not be used
223	The sign of an unsigned variable should not be tested
224	The standard "header file" "<csetjmp>" shall not be used
225	Unions should not be used
226	Unsigned "integer literals" shall be appropriately suffixed
227	"#import" should not be used
228	"#include" directives should only be preceded by preprocessor directives or comments
229	"#include" paths should be portable
230	"bool" expressions should not be used as operands to built-in operators other than =, &&, ||, !, ==, !=, unary &, and the conditional operator
231	"else" statements should be clearly matched with an "if"
232	"enum" members with implicit values should not collide with other enumerators
233	"enum" values should not be used as operands to built-in operators other than [ ], =, ==, !=, unary &, and the relational operators <, <=, >, >=
234	"for" loop stop conditions should be invariant
235	"Global variables" shall not be used
236	"goto" statement should not be used
237	"switch case" clauses should not have too many lines of code
238	"switch" statements should cover all cases
239	"switch" statements should not have too many "case" clauses
240	"Trigraph-like sequences" should not be used
241	#include directives in a file should only be preceded by other preprocessor directives or comments
242	A bit-field shall have an appropriate type
243	A cast should not convert a pointer type to an integral type
244	A function or object with external linkage should be "introduced" in a "header file"

245	A named function parameter shall be "used" at least once
246	A value should not be "unnecessarily written" to a local object
247	All the elements of an aggregate should be provided with an initial value
248	All uses of the #pragma directive should be documented
249	An "integer-literal" of type "long long" shall not use a single "L" or "l" in any suffix
250	An "object pointer type" shall not be cast to an integral type other than "std::uintptr_t" or "std::intptr_t"
251	Appropriate size arguments should be passed to "strncat" and "strlcpy"
252	Array declarations should include an explicit size specification
253	Assembly language should be encapsulated and isolated
254	Assignments should not be made from within conditions
255	Bit fields should be declared with appropriate types
256	Braces should be used to indicate and match the structure in the non-zero initialization of arrays and structures
257	Comma operator should not be used
258	Conditional operators should not be nested
259	Deprecated attributes should include explanations
260	Deprecated K&R syntax should not be used for function definition
261	Dereferenced null pointers should not be bound to references
262	Files should not have too many lines of code
263	Format strings should be used correctly
264	Functions should not be declared at block scope
265	Functions should not contain too many return statements
266	Functions should not have too many parameters
267	Functions that are not used in a project should be removed
268	Functions with "limited visibility" should be "used" at least once
269	Functions/methods should not have too many lines
270	GNU attributes should be used correctly
271	Identifiers should not be longer than 31 characters
272	If a function has internal linkage then all re-declarations shall include the static storage class specifer
273	Implicit casts should not lower precision
274	Include directives should not rely on non-portable search strategy
275	Increment (++) and decrement (--) operators should not be used in a method call or mixed with other operators in an expression
276	Increment should not be used to set boolean variables to 'true'
277	Keywords shall not be used as macros identifiers
278	Limited dependence should be placed on operator precedence
279	Lines should not be too long
280	Local variables should not be volatile
281	Loops should not have more than one "break" or "goto" statement
282	Magic numbers should not be used
283	Mergeable "if" statements should be combined
284	Multiline blocks should be enclosed in curly braces
285	Nested blocks of code should not be left empty
286	Obsolete POSIX functions should not be used
287	Operands of "&&" and "||" should be primary (C) or postfix (C++) expressions

288	Parameters in a function prototype should be named
289	Parameters should be passed in the correct order
290	Parentheses shall be used to ensure macro arguments are expanded appropriately
291	Redundant pairs of parentheses should be removed
292	Return value of "nodiscard" functions should not be ignored
293	Sections of code should not be "commented out"
294	Sections of code should not be commented out
295	Size argument of memory functions should be consistent
296	Size of bit fields should not exceed the size of their types
297	Statements should be on separate lines
298	Structures should not have too many fields
299	The "asm" declaration shall not be used
300	The "goto" statement should not be used
301	The argument to a "mixed-use macro parameter" shall not be subject to further expansion
302	The comma operator should not be used
303	The facilities provided by the standard "header file" "<csignal>" shall not be used
304	The result of an assignment operator should not be "used"
305	The ternary operator should not be used
306	The value of a complex expression should only be cast to a type that is narrower and of the same signedness as the underlying type of the expression
307	There shall be at most one occurrence of the # or ## operators in a single macro definition
308	Track parsing failures
309	Track uses of "FIXME" tags
310	Track uses of "NOSONAR" comments
311	Two branches in a conditional structure should not have exactly the same implementation
312	Types with "limited visibility" should be "used" at least once
313	Unevaluated operands should not have side effects
314	Unused assignments should be removed
315	Unused function parameters should be removed
316	Unused functions and methods should be removed
317	Unused labels should be removed
318	Unused type declarations should be removed
319	User-specified functions should not be called
320	Variables should not be shadowed
321	Within an enumerator list, the value of an implicitly-specified "enumeration constant" shall be unique
322	"#include_next" should not be used
323	"#pragma warning (default: ...)" should not be used
324	"/*" and "//" should not be used within comments
325	"^" should not be confused with exponentiation
326	"continue" should not be used
327	"if" statements should be preferred over "switch" when simpler
328	"Integral promotion" and the "usual arithmetic conversions" shall not change the signedness or the "type category" of an operand
329	"struct" names should comply with a naming convention
330	"union" names should comply with a naming convention
331	A "declaration" should not declare more than one variable or member variable

332	A "while" loop should be used instead of a "for" loop
333	A function should have a single point of exit at the end of the function
334	A single L in a literal suffix should only be used for long values
335	All "if ... else if" constructs shall be terminated with an "else" statement
336	All variables should be initialized
337	Appropriate char types should be used for character and integer values
338	Block scope "declarations" shall not be "visually ambiguous"
339	break statements should not be used except for switch cases
340	Code annotated as deprecated should not be used
341	Comment styles "//" and "/* ... */" should not be mixed within a file
342	Comments should not be located at the end of lines of code
343	Constants should come first in equality tests
344	Declarations should not be empty
345	Empty "case" clauses that fall through to the "default" should be omitted
346	Empty statements should be removed
347	Enumeration names should comply with a naming convention
348	Enumeration values should comply with a naming convention
349	Field names should comply with a naming convention
350	File names should comply with a naming convention
351	Files should end with a newline
352	Format strings should comply with ISO standards
353	Forward declarations should not be redundant
354	Function names should comply with a naming convention
355	Functions which do not return should be declared as "noreturn"
356	GNU extensions should not be used
357	Label names should comply with a naming convention
358	Lines should not end with trailing whitespaces
359	Literal suffix "L" for long integers shall be upper case
360	Local variable and function parameter names should comply with a naming convention
361	Loop variables should be declared in the minimal possible scope
362	Macro names should comply with a naming convention
363	Macros should not be redefined
364	Macros should not be used as replacements for "typedef" and "using"
365	Multicharacter literals should not be used
366	Multiple variables should not be declared on the same line
367	Nested code blocks should not be used
368	Non-standard attributes should not be used
369	Pointer and reference local variables should be "const" if the corresponding object is not modified
370	Pointer and reference parameters should be "const" if the corresponding object is not modified
371	Preprocessor operators "#" and "##" should not be used
372	Redundant casts should not be used
373	Signed and unsigned types should not be mixed in expressions
374	Source code should only use /* ... */ style comments
375	String literals should not be concatenated implicitly
376	Structure and union types should be complete at the end of a translation unit

377	Switch statement conditions should not have essentially boolean type
378	Tabulation characters should not be used
379	Tests of non-Boolean values against zero should be explicit
380	The "#" and "##" preprocessor operators should not be used
381	The "#pragma" directive and the "_Pragma" operator should not be used
382	The built-in unary "+" operator should not be used
383	The character sequence "/*" shall not be used within a C-style comment
384	The first element of an array should not be accessed implicitly
385	The lowercase form of "L" shall not be used as the first character in a literal suffix
386	The parameters in all "declarations" or overrides of a function shall either be unnamed or have identical names
387	The prefix increment/decrement form should be used
388	The same type aliases shall be used in all "declarations" of the same "entity"
389	The target type of a pointer or "lvalue" reference parameter should be const-qualified appropriately
390	The three expressions of a "for" statement should only be concerned with loop control
391	The value returned by a function shall be "used"
392	Track "TODO" and "FIXME" comments that do not contain a reference to a person
393	Type specifiers should be listed in a standard order
394	typedefs that indicate size and signedness should be used in place of the basic types
395	Types and variables should be declared in separate statements
396	Unused local variables should be removed
397	Variables with "limited visibility" should be "used" at least once
398	Deprecated code should be removed
399	Track comments matching a regular expression
400	Track instances of the "#error" preprocessor directive being reached
401	Track uses of "TODO" tags
402	"sprintf" should not be used
403	Changing directories improperly when using "chroot" is security-sensitive
404	Changing working directories without verifying the success is security-sensitive
405	Expanding archive files without controlling resource consumption is security-sensitive
406	Hard-coded passwords are security-sensitive
407	Hard-coded secrets are security-sensitive
408	Setting capabilities is security-sensitive
409	Setting loose POSIX file permissions is security-sensitive
410	Using "strcat" or "wcscat" is security-sensitive
411	Using "strcpy" or "wcscpy" is security-sensitive
412	Using "strlen" or "wcslen" is security-sensitive
413	Using "strncat" or "wcsncat" is security-sensitive
414	Using "strncpy" or "wcsncpy" is security-sensitive
415	Using "tmpnam", "tmpnam_s" or "tmpnam_r" is security-sensitive
416	Using clear-text protocols is security-sensitive
417	Using hardcoded IP addresses is security-sensitive
418	Using pseudorandom number generators (PRNGs) is security-sensitive
419	Using publicly writable directories is security-sensitive
420	Using weak hashing algorithms is security-sensitive